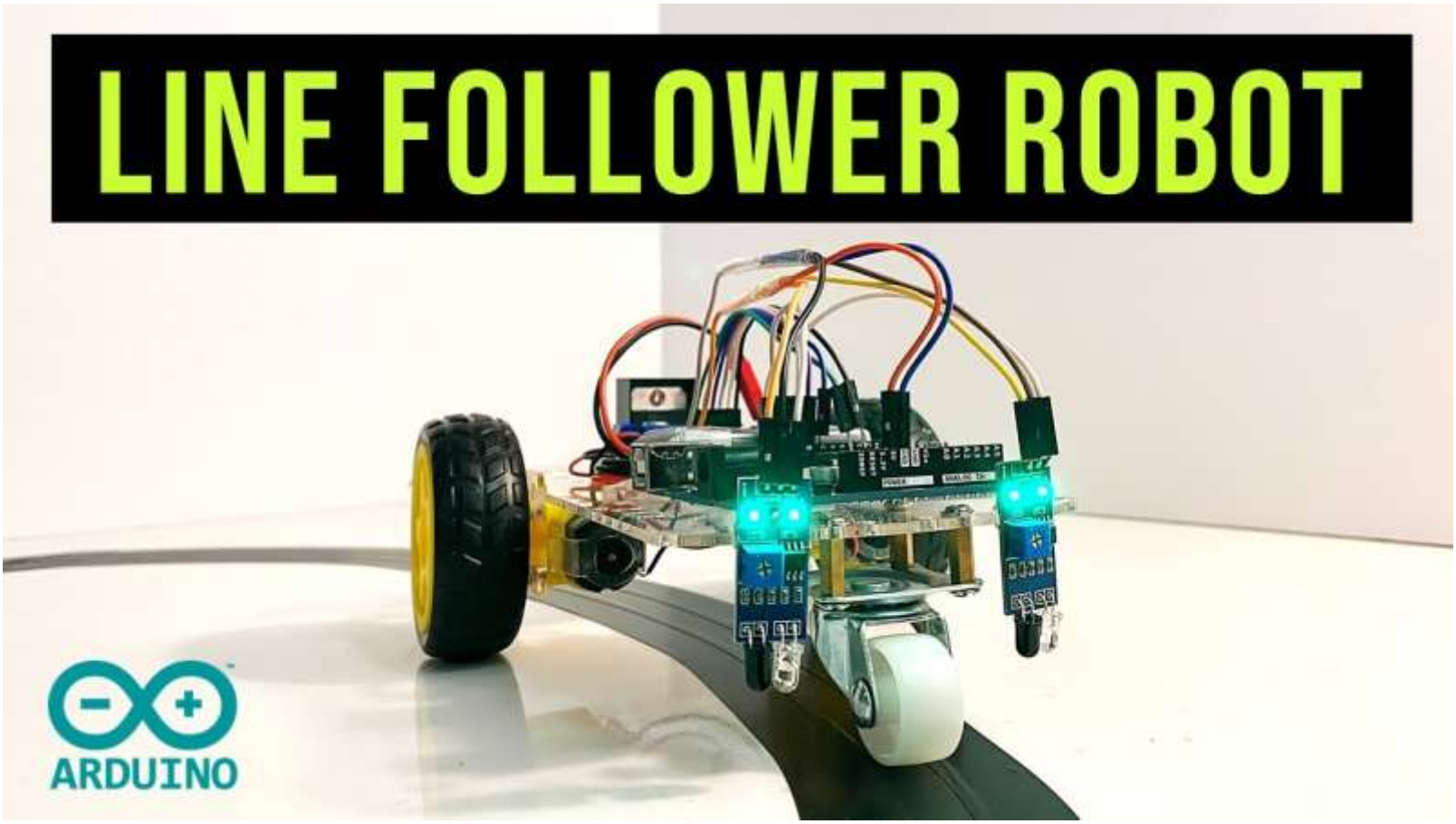
Line Follower



Made By: Abdelrahman Sherif Medhat

Code: 25033

Discription

This project involves a robotic line follower controlled by an Arduino Uno. The robot uses a 5-sensor infrared array to detect a black line on a light surface. A PID (Proportional-Integral-Derivative) control algorithm calculates an "error" value based on the sensor readings, which then adjusts the speed of the robot's two motors to keep it centered on the line. Additionally, the robot is equipped with a non-blocking ultrasonic sensor that continuously checks for obstacles. This allows the robot to halt its movement and avoid collisions without interrupting the core line-following logic.

What solution does it provide?

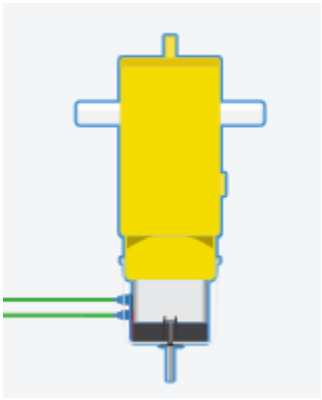
The main idea of this project is to create an autonomous robot that can navigate a course by following a pre-drawn line. The primary problem it solves is **guided movement and path-following** without human control.

The project tackles this in two main ways:

1. **Line Following with PID Control:** The robot uses a row of five infrared (IR) sensors to determine its position relative to a black line. The core of the solution is a **PID (Proportional-Integral-Derivative) controller**. This algorithm constantly calculates how far the robot is from the center of the line (the "error") and adjusts the speed of each motor to steer the robot back to the correct path. This ensures the robot follows the line smoothly and doesn't just wander aimlessly.
2. **Obstacle Avoidance:** The project also solves the problem of **collision avoidance**. It uses an ultrasonic sensor to measure the distance to any objects in front of the robot. The code is designed to be non-blocking, meaning the robot can check for obstacles in the background without stopping the line-following logic. If the ultrasonic sensor detects an object too close, the robot stops to prevent a crash.

Components

Hobby Gear Motor



9V Battery



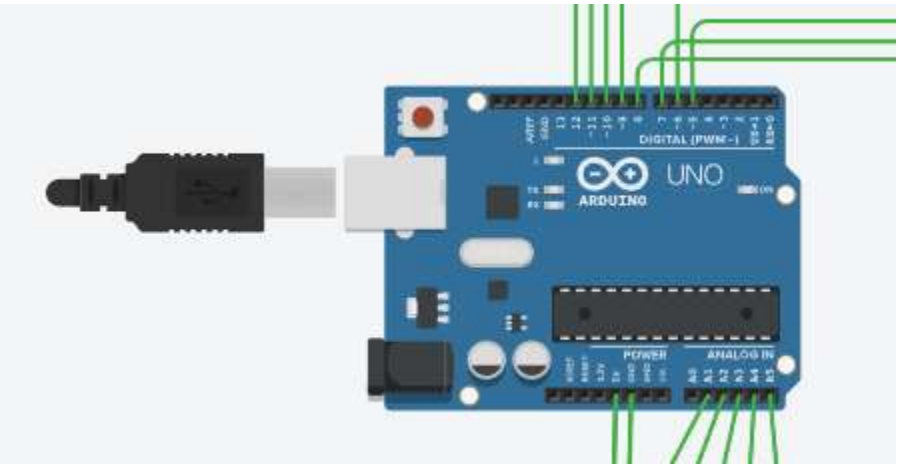
H-Bridge Motor Driver (L293D)



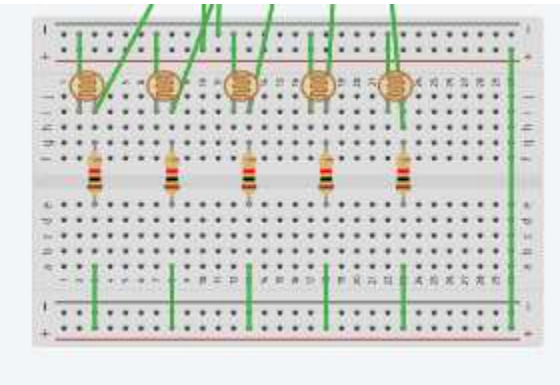
Ultrasonic Sensor



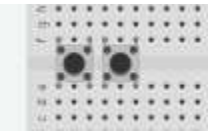
Arduino UNO



InfraRed Sensor Array



Push Buttons



Main Problems Encountered

1. PID Tuning and Oscillation

The most common challenge is tuning the PID constants (Kp, Ki, and Kd). The placeholder values in the code are just a starting point. If the values are not correctly tuned for your

specific robot's weight, motor power, and the track's surface, the robot will behave erratically:

- **High Kp:** Causes the robot to over-correct and oscillate wildly around the line.
- **High Kd:** Can make the robot jittery or over-dampen its turns.
- **Ki (Integral term):** If used incorrectly, this can cause the robot to overshoot corners and wander off the line.

2. Sensor Calibration Issues

While the calibration function is designed to handle different lighting conditions, its effectiveness depends on the environment. If the robot's spinning motion during calibration doesn't expose all sensors to both the dark line and the light background, the calculated midpoint thresholds will be incorrect. This can lead to the robot not reacting to the line at all, as it's consistently misinterpreting the sensor readings.

3. Mechanical and Hardware Flaws

The code assumes a perfect mechanical setup, but real-world issues often arise.

- **Uneven Motors:** Even at the same analogWrite() value, motors may spin at slightly different speeds, causing the robot to drift.
- **Sensor Spacing:** If the sensors are spaced too far apart, the robot may fail to detect the line on sharp turns, leading to it getting lost.

4. Non-Blocking Logic and Timing

The use of the millis() function for non-blocking obstacle detection is crucial, but it introduces a new set of problems.

- **Timing:** If the PING_INTERVAL is too long, the robot might not detect an obstacle in time to stop. If it's too short, the frequent calculations could slow down the main line-following loop.

Program code

1. Pin Definitions and Constants

This section defines the hardware connections and the core tuning parameters for the project. Using #define and const makes the code readable and easy to modify.

```
//motor A is the right wheel
// Motor Driver Pins
#define ENA 5  // Motor A Enable
#define IN1 7  // Motor A Input 1
#define IN2 8  // Motor A Input 2
#define IN3 9  // Motor B Input 1
#define IN4 10 // Motor B Input 2
#define ENB 6  // Motor B Enable

// Sensor Pins
#define SENSOR_LEFTMOST  A1
#define SENSOR_LEFT      A2
#define SENSOR_CENTER    A3
#define SENSOR_RIGHT     A4
#define SENSOR_RIGHTMOST A5
const int SENSOR_PINS[] = {SENSOR_LEFTMOST, SENSOR_LEFT, SENSOR_CENTER, SENSOR_RI
const int NUM_SENSORS = 5;

#define TRIG_PIN 11
#define ECHO_PIN 12

// Ultrasonic sensor variables
long lastPingTime = 0;
const int PING_INTERVAL = 100; // Time in milliseconds between pings
const int OBSTACLE_DISTANCE_CM = 100; // Stop if obstacle is closer than this

// PID Constants
// P (Proportional): Adjusts the motor speed based on the current error.
// I (Integral): Accounts for past errors, helping to eliminate steady-state erro
// D (Derivative): Predicts future error, damping oscillations.
double Kp = 1;
double Ki = 0;
double Kd = 0.5;
```

```
// PID variables
double lastError = 0;
double integral = 0;

// Base speed for the motors
int baseSpeed = 150; // Adjust this value between 0-255

// Sensor calibration variables
int sensorMin[NUM_SENSORS];
int sensorMax[NUM_SENSORS];
int sensorThresholds[NUM_SENSORS]; // Array to store individual thresholds

```

Function: This code establishes the specific digital and analog pins on the Arduino that are connected to the motor driver, IR sensors, and ultrasonic sensor. It also defines the PID constants, which are the main variables you would tune to control the robot's line-following behavior.

2. The setup() Function

The setup() function is a standard part of all Arduino sketches. It runs only once when the program starts.

```
48  void setup() {
49    Serial.begin(9600);
50    pinMode(ENA, OUTPUT);
51    pinMode(IN1, OUTPUT);
52    pinMode(IN2, OUTPUT);
53    pinMode(IN3, OUTPUT);
54    pinMode(IN4, OUTPUT);
55    pinMode(ENB, OUTPUT);
56
57    // Set LINE FOLLOWER FORWARD
58    digitalWrite(IN1, HIGH);
59    digitalWrite(IN2, LOW);
60    digitalWrite(IN3, HIGH);
61    digitalWrite(IN4, LOW);
62
63    // Configure ultrasonic sensor pins
64    pinMode(TRIG_PIN, OUTPUT);
65    pinMode(ECHO_PIN, INPUT);
66
67    // Calibrate sensors
68    calibrateSensors();
69  }
```

Function: This section initializes the serial communication for debugging, configures all motor and sensor pins as either INPUT or OUTPUT, sets the initial motor direction to move forward, and crucially, calls the calibrateSensors() function to prepare the robot for the specific lighting and surface conditions.

3. Sensor Calibration

This function is a key part of the project. It makes the robot spin in a circle to measure the minimum (light surface) and maximum (dark line) readings for each individual sensor.

```
  // Calibrate sensors
  calibrateSensors();
}

// Function to perform sensor calibration
void calibrateSensors() {
  Serial.println("Calibrating sensors...");

  // Initialize min/max arrays with extreme values
  for (int i = 0; i < NUM_SENSORS; i++) {
    sensorMin[i] = 1023;
    sensorMax[i] = 0;
  }

  // Rotate the robot to read sensor values
  // Turn left to expose sensors to both the line and the background
  analogWrite(ENA, baseSpeed);
  analogWrite(ENB, 0); // Stops the left motor to turn right
  delay(1000); // we can adjust the calibration speed

  // Keep turning and reading for a while
  long startTime = millis();
  while (millis() - startTime < 3000) { // Rotate for 3 seconds
    for (int i = 0; i < NUM_SENSORS; i++) {
      int sensorValue = analogRead(SENSOR_PINS[i]);
      if (sensorValue < sensorMin[i]) {
        sensorMin[i] = sensorValue;
      }
      if (sensorValue > sensorMax[i]) {
        sensorMax[i] = sensorValue;
      }
    }
  }
```

Function: The function finds a unique sensorThresholds value for each sensor by averaging the highest and lowest readings it sees during the rotation. This makes the robot's performance more consistent across different environments.

4. Sensor Reading and Error Calculation

This function is responsible for translating the sensor data into a single "error" value that the PID controller can use.

```
// Function to read sensor values and determine the error
int readSensors() {
  // Read analog values from the sensors
  int s1 = lineDetected(0, analogRead(SENSOR_LEFTMOST));
  int s2 = lineDetected(1, analogRead(SENSOR_LEFT));
  int s3 = lineDetected(2, analogRead(SENSOR_CENTER));
  int s4 = lineDetected(3, analogRead(SENSOR_RIGHT));
  int s5 = lineDetected(4, analogRead(SENSOR_RIGHTMOST));

  // Weights for error calculation
  // A positive error means on the left.
  //A negative error means on the right.
  int error = s1 * -2 + s2 * -1 + s3 * 0 + s4 * 1 + s5 * 2;

  return error;
}
```

Function: It first calls lineDetected() for each sensor to determine if it is over the line. Then, it uses a weighted sum to calculate the error. An error of 0 means the robot is perfectly centered, while a positive or negative error indicates it has drifted left or right.

5. Main Control Loop

The loop() function is where all the action happens. It runs continuously and orchestrates the robot's movement.

```
void loop() {
  float distance = readUltrasonicSensor();

  // Check for a valid distance reading from the ultrasonic sensor
  if (distance != -1.0 && distance < OBSTACLE_DISTANCE_CM) {
    // If an obstacle is detected, stop the robot
    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
    Serial.println("OBSTACLE DETECTED! STOPPING!");
  } else {
    // If no obstacle is detected or if it is not time to ping,
    // continue with the line-following logic
    int error = readSensors();

    // Proportional term
    double proportional = error * Kp;

    // Integral term
    integral = integral + error;
    double integralTerm = integral * Ki;

    // Derivative term
    double derivative = error - lastError;
    double derivativeTerm = derivative * Kd;
    lastError = error;

    // PID output with multiplier
    double pidOutput = proportional + integralTerm + derivativeTerm;

    // Calculate motor speeds
    int motorSpeedA = baseSpeed - 25 * pidOutput;
    int motorSpeedB = baseSpeed + 25 * pidOutput;

    // Clamp motor speeds to the valid range (0-255)
    motorSpeedA = constrain(motorSpeedA, 0, 255);
    motorSpeedB = constrain(motorSpeedB, 0, 255);

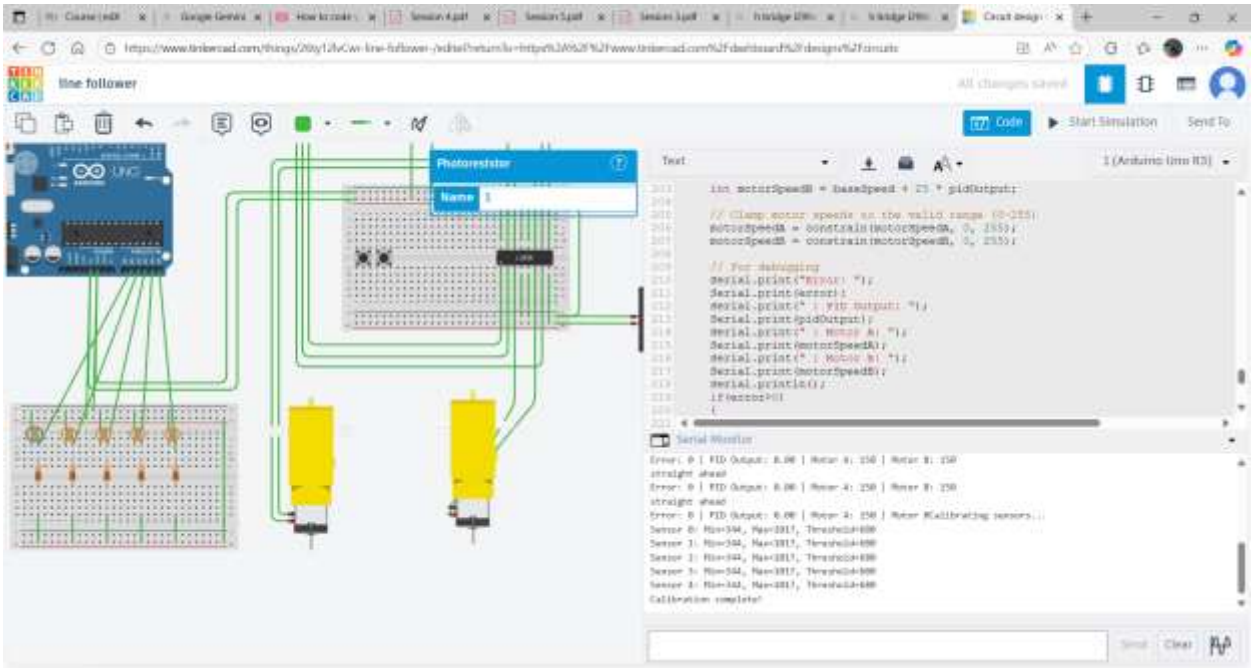
    // For debugging
    Serial.print("Error: ");
    Serial.print(error);
    Serial.print(" | PID Output: ");
    Serial.print(pidOutput);
    Serial.print(" | Motor A: ");
    Serial.print(motorSpeedA);
    Serial.print(" | Motor B: ");
    Serial.print(motorSpeedB);
    Serial.println();
    if(error>0)
    {
      Serial.println("Right turn");
    }
    else if(error<0)
    {
      Serial.println("Left turn");
    }
    else if(error==0)
    {
      Serial.println("straight ahead");
    }

    // Write speeds to motors
    analogWrite(ENA, motorSpeedA);
    analogWrite(ENB, motorSpeedB);
  }
}
```

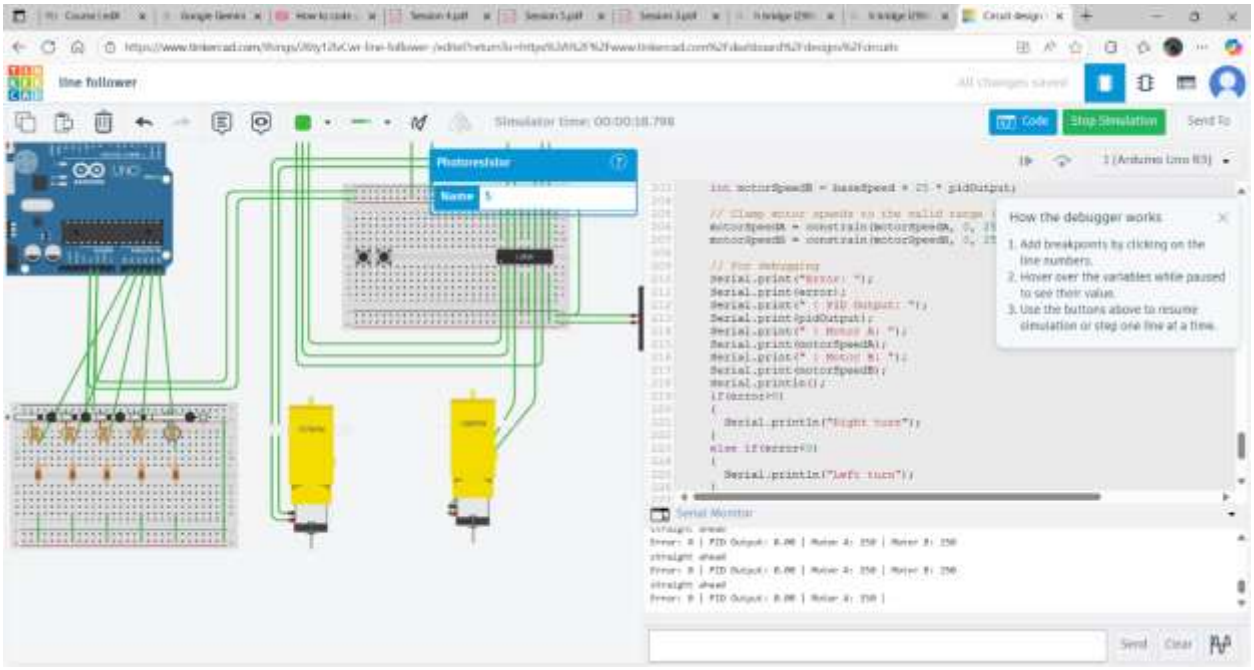
Function: This section first checks the ultrasonic sensor to see if there is an obstacle. If there is, it stops the robot. If not, it executes the line-following logic: calculating the PID output from the error value and adjusting the motor speeds accordingly. This ensures the robot stays on the line while also avoiding collisions.

Simulation Snippets

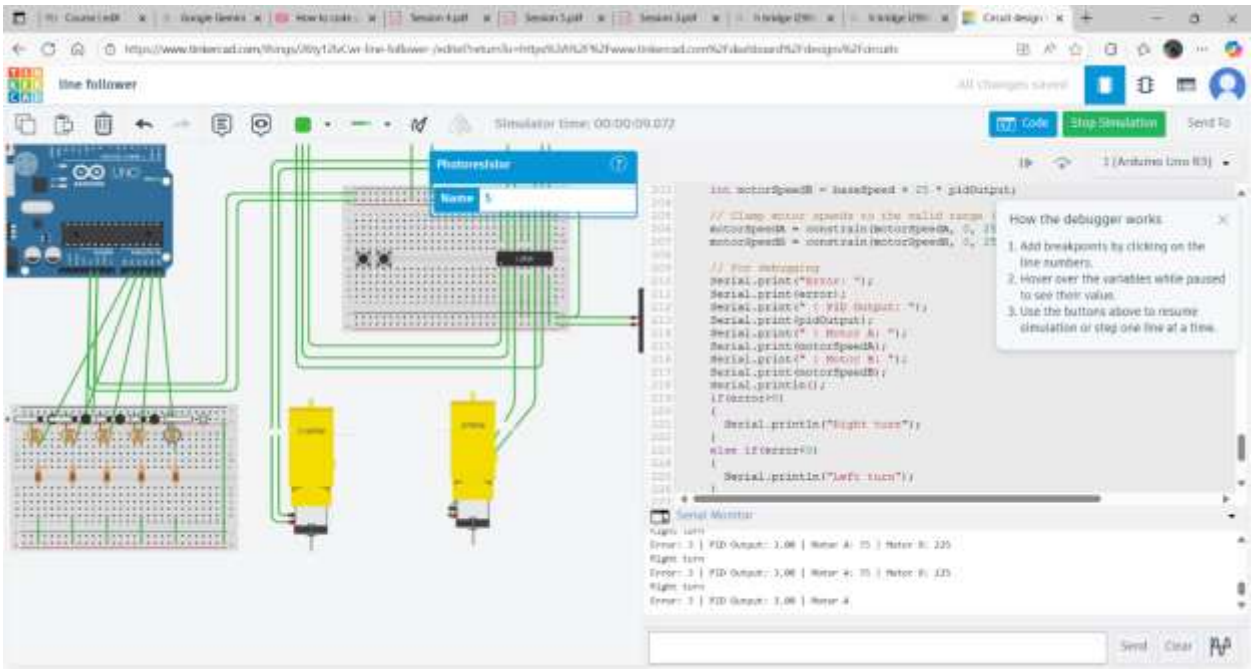
Sensor Calibration



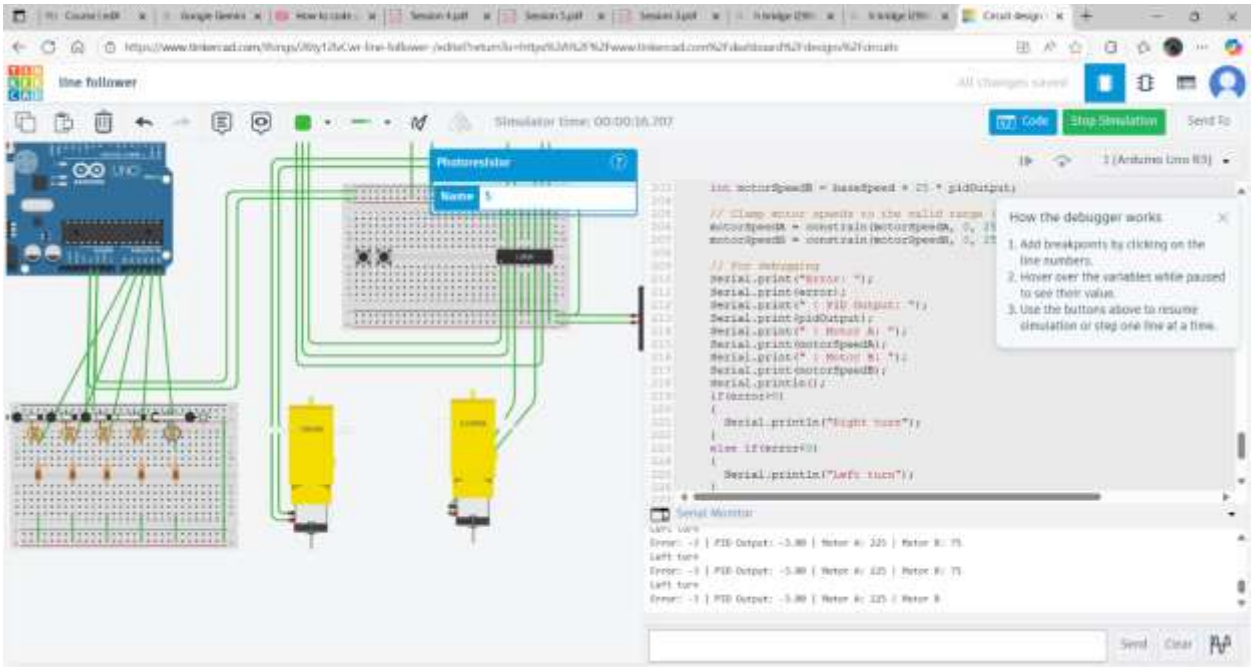
Straight Ahead



Right Turn



Left Turn



Obstacle Detection

